

Online Streaming Platform – System Architecture & Security Design (Updated)

1. Project Overview

This document describes the system architecture, project structure, DevOps approach, and security measures for an online streaming platform similar to Netflix. The platform will include web and mobile clients built with React and optionally TypeScript, a backend API layer, integration with TMDB for movie data, and NewsAPI for movie-related news articles.

2. High-Level Architecture

```
React / React Native (TypeScript optional)
  |
  v
Backend API (FastAPI / Node.js)
  |
  ├── TMDB API (Movie Metadata)
  ├── NewsAPI (Movie News & Articles)
  ├── Supabase Auth & PostgreSQL
  └── Redis Cache
```

- Frontend communicates only with the backend API. - Backend securely fetches data from TMDB and NewsAPI. - Redis caching used to minimize API calls. - Supabase handles user authentication, watchlists, and history.

3. Frontend Project Structure (React + TypeScript)

```
src/
├── components/      # Reusable UI components
├── pages/           # Route-based pages (Home, Details, Profile, News)
├── hooks/           # Custom React hooks
├── services/        # API communication layer (TMDB, NewsAPI, Supabase)
├── context/         # Global state management (Context API / Zustand /
Redux)
├── types/           # TypeScript interfaces and types
└── styles/          # Global styles and TailwindCSS configuration
```

Key Pages: - Home (Trending Movies, Popular Movies, Movie News) - Movie Details (Overview, Cast, Trailers, Related News) - Search (Movies & TV Shows) - Watchlist - Profile / Account

4. Backend Project Structure

```

app/
├─ main.py           # Entry point
├─ api/              # Route definitions
├─ services/         # TMDB, NewsAPI, business logic
├─ models/           # Database models (Supabase/PostgreSQL)
├─ auth/             # Authentication and authorization
├─ cache/            # Redis caching logic
└─ utils/            # Helper functions (API normalization, error handling)

```

Endpoints: - `/api/movies/trending` - `/api/movies/:id` - `/api/search` - `/api/watchlist`
 (add/remove/get) - `/api/news/trending` - `/api/news/movie/:tmdb_id`

5. Data Flow

1. Client sends request to backend API.
2. Backend authenticates request via Supabase JWT.
3. Backend fetches TMDB data and/or NewsAPI data.
4. Backend caches API responses in Redis.
5. Backend returns sanitized JSON response to frontend.
6. Frontend renders data in React components.

6. DevOps & Deployment

- Docker for containerization
- GitHub Actions for CI/CD (lint, test, build, deploy)
- Hosting: Vercel (frontend), Render / Railway (backend)
- Environment variables for secrets (TMDB API key, NewsAPI key, Supabase credentials)
- Optional monitoring/logging setup

7. Security Measures

7.1 API Security - JWT authentication via Supabase - Role-based access control - Rate limiting using Redis

7.2 Data Security - HTTPS enforced for all endpoints - Secrets stored in environment variables - Database encryption at rest (Supabase)

7.3 Third-Party API Protection - TMDB and NewsAPI keys stored only server-side - Backend proxy prevents direct key exposure

7.4 Application Security - CORS configuration - Input validation and sanitization - Protection against XSS, CSRF

8. Movie News Feature

- **News Source:** NewsAPI.org
- **Endpoints:**

- `/api/news/trending` – fetch latest movie-related news
- `/api/news/movie/:tmdb_id` – fetch news specific to a movie
- **Caching:** Redis (TTL = 30–60 minutes)
- **Frontend:** React components display article title, source, publish date, thumbnail, external link
- **Legal:** Always link back to original publisher, do not store full article content

9. Scalability Considerations

- Stateless backend services allow horizontal scaling
- Redis caching reduces API calls and improves response times
- Supabase handles database scaling and auth securely
- Frontend built for responsiveness and modularity, compatible with React Native for mobile

10. Technology Stack Summary

Layer	Technology
Frontend	React.js + TypeScript (optional), TailwindCSS
Backend	FastAPI / NestJS
Movie Data	TMDB API
News Data	NewsAPI.org
Auth & DB	Supabase (PostgreSQL)
Cache	Redis (Upstash / self-hosted)
Hosting	Vercel (frontend), Render/Railway (backend)
CI/CD	GitHub Actions

11. Notes

- Backend handles all third-party API integrations and caching.
- Frontend focuses on UI/UX and consuming backend APIs.
- Supabase provides secure, managed auth and relational data storage.
- News feature is integrated without storing full articles to comply with copyright.
- MVP timeline: 4 weeks with initial pages and news integration.